

Exercícios OAC

Hierarquia de Memória - Respostas

5.1.1 Quantos inteiros de 32 bits podem ser armazenados em um bloco de cache de 16 bytes?

Lembrando que um inteiro de 32 bits ocupa 4 bytes:

$$\text{Número de inteiros} = \text{Tamanho do bloco} / \text{Tamanho do inteiro} = 16 / 4 = 4$$

5.1.2 Quais referências de variáveis exibem localidade temporal?

A localidade temporal ocorre quando uma variável é acessada repetidamente em um curto período de tempo. No código fornecido:

- I e J são acessadas repetidamente no loop
- $B[i][0]$ é acessada repetidamente, uma vez que para cada valor de I temos 8000 acessos (J)

5.1.3 Quais referências de variáveis exibem localidade espacial?

A **localidade espacial** ocorre quando elementos de memória com endereços próximos são acessados sequencialmente. Neste caso, $A[i][j]$ apresenta localidade espacial.

A localidade é afetada pela ordem de referência e pelo layout dos dados. O mesmo cálculo também pode ser escrito abaixo em Matlab, que difere de C porque **armazena elementos de matriz dentro da mesma coluna de forma contígua na memória**.

```
for I=1:8
    for J=1:8000
        A(I,J)=B(I,0)+A(J,I);
    end
end
```

5.1.4 Quais referências de variáveis exibem localidade temporal?

- I e J são acessadas repetidamente no loop
- $B[i][0]$ é acessada repetidamente, uma vez que para cada valor de I temos 8000 acessos (J)

5.1.5 Quais referências de variáveis exibem localidade espacial?

Como os inteiros são armazenados por colunas, $B[i][0]$ exibe localidade espacial, assim como $A[j][i]$

5.2.1 Tabela com as respostas

Word Address	Binary Address	Tag	Index	Hit/Miss
0x03	0000 0011	0	3	M
0xb4	1011 0100	b	4	M
0x2b	0010 1011	2	b	M
0x02	0000 0010	0	2	M
0xbf	1011 1111	b	f	M
0x58	0101 1000	5	8	M
0xbe	1011 1110	b	e	M
0x0e	0000 1110	0	e	M
0xb5	1011 0101	b	5	M
0x2c	0010 1100	2	c	M
0xba	1011 1010	b	a	M
0xfd	1111 1101	f	d	M

5.2.2 Tabela com as respostas

Word Address	Binary Address	Tag	Index	Offset	Hit/Miss
0x03	0000 0011	0	1	1	M
0xb4	1011 0100	b	2	0	M
0x2b	0010 1011	2	5	1	M
0x02	0000 0010	0	1	0	H
0xbf	1011 1111	b	7	1	M
0x58	0101 1000	5	4	0	M
0xbe	1011 1110	b	7	0	H
0x0e	0000 1110	0	7	0	M
0xb5	1011 0101	b	2	1	H
0x2c	0010 1100	2	6	0	M
0xba	1011 1010	b	5	0	M
0xfd	1111 1101	f	6	1	M

5.2.3 Tabela comparativa

Word Address	Binary Address	Tag	Cache 1		Cache 2		Cache 3	
			Index	Hit/miss	Index	Hit/miss	Index	Hit/miss
0x03	0000 0011	0x00	3	M	1	M	0	M
0xb4	1011 0100	0x16	4	M	2	M	1	M
0x2b	0010 1011	0x05	3	M	1	M	0	M
0x02	0000 0010	0x00	2	M	1	M	0	M
0xbf	1011 1111	0x17	7	M	3	M	1	M
0x58	0101 1000	0x0b	0	M	0	M	0	M
0xbe	1011 1110	0x17	6	M	3	H	1	H
0x0e	0000 1110	0x01	6	M	3	M	1	M
0xb5	1011 0101	0x16	5	M	2	H	1	M
0x2c	0010 1100	0x05	4	M	2	M	1	M
0xba	1011 1010	0x17	2	M	1	M	0	M
0xfd	1111 1101	0x1F	5	M	2	M	1	M

Cache 1 miss rate = 100%

Cache 2 miss rate = $10/12 = 83\%$

Cache 3 miss rate = $11/12 = 92\%$

Cache 2 tem a menor taxa de erros.

5.3.1 Total de bits

Bloco = 2 words

Número de entradas da cache: $32\text{KB}/2 \text{ words} = 32\text{KB} / 8\text{B} = 4\text{K} = 4096 \text{ entradas}$

Offset byte na palavra: 2 bits

Offset da palavra no bloco: 1 bit

Entrada da cache: 12 bits

TAG: 17 bits

Bit de validade: 1 bit

Total de bits da cache: $4096 \times (1 + 17 + 64) = 335872 \text{ bits}$

5.3.2 Total de bits

Número de entradas da cache (bloco = 16 words)

$64\text{KB}/16 \text{ words} = 64\text{KB} / 64\text{B} = 1024 \text{ entradas}$

Offset byte na palavra: 2 bits

Offset da palavra no bloco: 4 bits

Entrada da cache: 10 bits

TAG: 16 bits

Bit de validade: 1 bit

Total de bits da cache: $1024 \times (1 + 16 + 512) = 541696 \text{ bits}$ (menos que o dobro anterior)

5.9.1 Para latência de $20 \times B$:

- $B = 8$: $\text{Lat} = 0.040 \times (20 \times 8) = 6.40$
- $B = 16$: $\text{Lat} = 0.030 \times (20 \times 16) = 9.60$
- $B = 32$: $\text{Lat} = 0.020 \times (20 \times 32) = 12.80$
- $B = 64$: $\text{Lat} = 0.015 \times (20 \times 64) = 19.20$
- $B = 128$: $\text{Lat} = 0.010 \times (20 \times 128) = 25.60$

$B = 8$ é a melhor solução.

5.9.2 Para latência de $24 + B$:

- $B = 8$: $\text{Lat} = 0.040 \times (24 + 8) = 1.28$
- $B = 16$: $\text{Lat} = 0.030 \times (24 + 16) = 1.20$
- $B = 32$: $\text{Lat} = 0.020 \times (24 + 32) = 1.12$
- $B = 64$: $\text{Lat} = 0.015 \times (24 + 64) = 1.32$
- $B = 128$: $\text{Lat} = 0.010 \times (24 + 128) = 1.52$

$B = 32$ é a melhor solução.

5.9.3 Para latência constante (supor K)

- $B = 8$: $\text{Lat} = 0.040 \times K$
- $B = 16$: $\text{Lat} = 0.030 \times K$
- $B = 32$: $\text{Lat} = 0.020 \times K$
- $B = 64$: $\text{Lat} = 0.015 \times K$
- $B = 128$: $\text{Lat} = 0.010 \times K$

$B = 128$ tem a menor taxa de falhas, sendo a melhor solução.

5.10.1 Taxas de clock:

P1: 1.515 GHz

P2: 1.11 GHz

5.10.2 AMAT de P1 e P2:

P1: 6.31 ns = 9.56 ciclos

P2: 5.11 ns = 5.68 ciclos

Para P1, todos os acessos à memória exigem pelo menos um ciclo (para acessar L1). 8% dos acessos à memória exigem adicionalmente um acesso de 70 ns à memória principal. Isso é $70/0,66 = 106,06$ ciclos. No entanto, não podemos dividir os ciclos; portanto,

devemos arredondar para 107 ciclos. Portanto, o tempo médio de acesso à memória é $1 + 0,08 \cdot 107 = 9,56$ ciclos, ou 6,31 ps.

Para P2, um acesso à memória principal leva 70 ns. Isso é $70/0,90 = 77,78$ ciclos. Como não podemos dividir os ciclos, devemos arredondar para 78 ciclos. Portanto, o tempo médio de acesso à memória é $1 + 0,06 \cdot 78 = 5,68$ ciclos, ou 6,11 ps.

5.10.2 CPI de P1 e P2:

P1: 12.64 CPI = 8.34 ns / instrução

P2: 7.35 CPI = 6.63 ns / instrução

Para P1, cada instrução requer pelo menos um ciclo. Além disso, 8% de todas as instruções faltam na *cache* de instruções e incorrem em um atraso de 107 ciclos. Além disso, 36% das instruções são acessos a dados. Oito por cento desses 36% são falhas de *cache*, o que adiciona 107 ciclos adicionais.

$$1 + .08 \cdot 107 + .36 \cdot .08 \cdot 107 = 12,64$$

Com um ciclo de clock de 0,66 ps, cada instrução requer 8,34 ns.

Usando a mesma lógica, podemos ver que P2 tem um CPI de 7,36 e uma média de apenas 6,63 ns/instrução.